



Ejercicios propuestos

NIVEL BÁSICO

Enfoque: lógica de control (if/switch/loops), variables y manipulación de cadenas. Todo puede hacerse dentro del método Main o con métodos estáticos simples.

Ejercicio 1: El simulador de batalla RPG

Objetivo: controlar un bucle while complejo y usar la clase Random.

Escenario: el estudiante debe crear un minijuego de consola donde un héroe lucha contra un monstruo hasta que uno de los dos muere.

Instrucciones detalladas:

Configuración inicial:

El héroe inicia con 50 puntos de vida (HP).

El monstruo inicia con 50 HP.

Define una poción que cura 10 HP (pero solo se puede usar 3 veces).

Flujo del turno (dentro de un bucle):

En cada turno, el usuario elige: 1. Atacar o 2. Curarse.

Si ataca: genera un número aleatorio entre 5 y 12. Resta eso a la vida del monstruo.

Si se cura: suma 10 a su vida (sin pasar de 50) y resta 1 al contador de pociones. Si no quedan pociones, pierde el turno.

Turno del monstruo: después de la acción del jugador, el monstruo ataca automáticamente (daño aleatorio entre 7 y 15).

Fin del juego:

El bucle termina cuando la vida de alguno llega a 0 o menos.

Imprimir mensaje de "¡Victoria!" o "Has muerto...".



Ejercicio 2: validador de contraseñas seguro

Objetivo: manipulación de Strings y validación lógica.

Escenario: crear un sistema que pida al usuario crear una contraseña y no lo deje avanzar hasta que cumpla reglas estrictas.

Instrucciones detalladas:

Entrada de datos:

Solicitar al usuario que ingrese una contraseña.

Reglas de validación (debe cumplir **TODAS**):

Tener al menos 12 caracteres.

Contener al menos una mayúscula (`char.IsUpper`).

Contener al menos un número (`char.IsDigit`).

No contener la palabra "password" ni "123" (independientemente de mayúsculas/minúsculas).

Feedback inmediato:

Si la contraseña falla, el programa debe decir exactamente qué regla falló (ej: "Falta un número").

El programa debe volver a pedir la contraseña hasta que sea válida.

Confirmación:

Una vez válida, pedir que la ingrese nuevamente para confirmar. Si no coinciden, reiniciar el proceso.

NIVEL INTERMEDIO

Enfoque: programación orientada a objetos (clases, propiedades), listas/colas y separación de responsabilidades.

Ejercicio 3: Sistema de gestión de empleados (CRUD en memoria)

Objetivo: uso de clases, List<T> y LINQ básico (Find/Remove).

Instrucciones detalladas:

La clase empleado:

Propiedades: id (string), nombre (string), puesto (string), salario (decimal).

El menú:

Crear un menú que se repita hasta elegir "Salir".

1. Agregar: pide datos. validación: el id no puede repetirse. El salario no puede ser negativo.
2. Listar todos: muestra una lista formateada (ej: [id: 001] Juan - gerente - \$5000).
3. Buscar: pide un id y muestra los datos de ese empleado específico.
4. Eliminar: pide un id y borra al empleado de la lista. Mostrar mensaje si el id no existe.

Requisito de código:

Separar la lógica: la clase empleado no debe tener Console.WriteLine. Todo lo visual va en Program.cs.

Ejercicio 4: control de turnos bancarios (colas)

Objetivo: uso de la estructura de datos Queue<T> (FIFO - primero en entrar, primero en salir).

Escenario: simular una pantalla de turnos de un banco.

Instrucciones detalladas:

La clase cliente:

Propiedades: NumeroTurno (int, autoincremental) y Nombre (string).

Gestión de la cola:

Usar un Queue<Cliente> para los clientes en espera.

Operaciones:

Opción 1: llegada del cliente: pide el nombre, le asigna el siguiente número de turno disponible y lo agrega a la cola. Imprime: "Bienvenido [Nombre], su turno es [N]".

Opción 2: atender siguiente: saca al primer cliente de la cola (Dequeue). muestra: "Atendiendo al turno [N] - [Nombre]". Si la cola está vacía, mostrar mensaje de error.

Opción 3: estado de la cola: muestra cuántos clientes hay esperando y quién es el siguiente (sin sacarlo, usando Peek).



NIVEL DIFÍCIL

Enfoque: persistencia de datos (archivos), manejo de excepciones, LINQ avanzado y arquitectura robusta.

Ejercicio 5: analizador de logs del servidor (archivos + LINQ)

Objetivo: leer archivos reales, procesar strings ("parsing") y generar reportes.

Preparación: el estudiante debe crear manualmente un archivo de texto llamado logs.txt con este formato (unas 10 líneas): INFO|2023-10-01|Inicio de sistema ERROR|2023-10-01|Fallo de conexión DB WARNING|2023-10-02|Memoria alta ERROR|2023-10-02|NullReferenceException

Instrucciones detalladas:

Lectura:

Crear una clase LogEntry (tipo, fecha, mensaje).

Leer el archivo línea por línea usando StreamReader.

Usar String.Split('|') para separar los datos y convertirlos a objetos LogEntry. Guardarlos en una List<LogEntry>.

Manejo de errores: usar try-catch por si el archivo no existe o una línea tiene formato incorrecto (saltar esa línea, no cerrar el programa).

Análisis (LINQ):

Contar cuántos errores ("ERROR") hubo en total.

Agrupar los logs por tipo (Cuántos INFO, cuántos WARNING, cuántos ERROR).

Escritura:

Crear un nuevo archivo llamado resumen.txt usando StreamWriter.

Escribir dentro el resultado del análisis (ej: "Total Errores: 2").



Ejercicio 6: gestor de tareas con persistencia JSON

Objetivo: serialización JSON, uso de fechas y actualización de estados.

Escenario: una "To-Do List" que no pierde los datos cuando cierras la consola.

Instrucciones detalladas:

Dependencias: usar System.Text.Json (nativo en .NET Core/5+).

Clase tarea:

Título (string), FechaLimite (DateTime), EsCompletada (bool).

Persistencia (Guardar/Cargar):

Al iniciar el programa, verificar si existe tareas.json. Si existe, cargarlo y deserializarlo en una lista. Si no, iniciar lista vacía.

Al seleccionar "Salir", serializar la lista y guardar los cambios en tareas.json.

Funcionalidad:

Agregar tarea: pedir título y fecha (validar que la fecha no sea en el pasado).

Marcar completada: mostrar lista de pendientes, seleccionar una por índice y cambiar EsCompletada a true.

Ver pendientes: mostrar solo las tareas donde EsCompletada == false. Si la FechaLimite ya pasó, mostrarla en color ROJO (Console.ForegroundColor).